

Transformer Explainer — A Backend Engineering Tour

A walkthrough of the codebase for a junior software engineer learning real-world backend development.

Why this report exists

The transformer-explainer is a public web app at transformer-explainer-three.vercel.app. Its **content** is an interactive tutorial on how Transformer language models work internally. But the **codebase** is a deliberately legible example of a modern full-stack TypeScript app — the kind a junior backend engineer would meet on day one of a real job.

This report walks through the system the way a senior engineer would explain it on a whiteboard: starting from a single HTTP request, drilling into the layers it touches, then zooming back out to the production infrastructure and the failure modes we actually hit while building it.

You should already know:

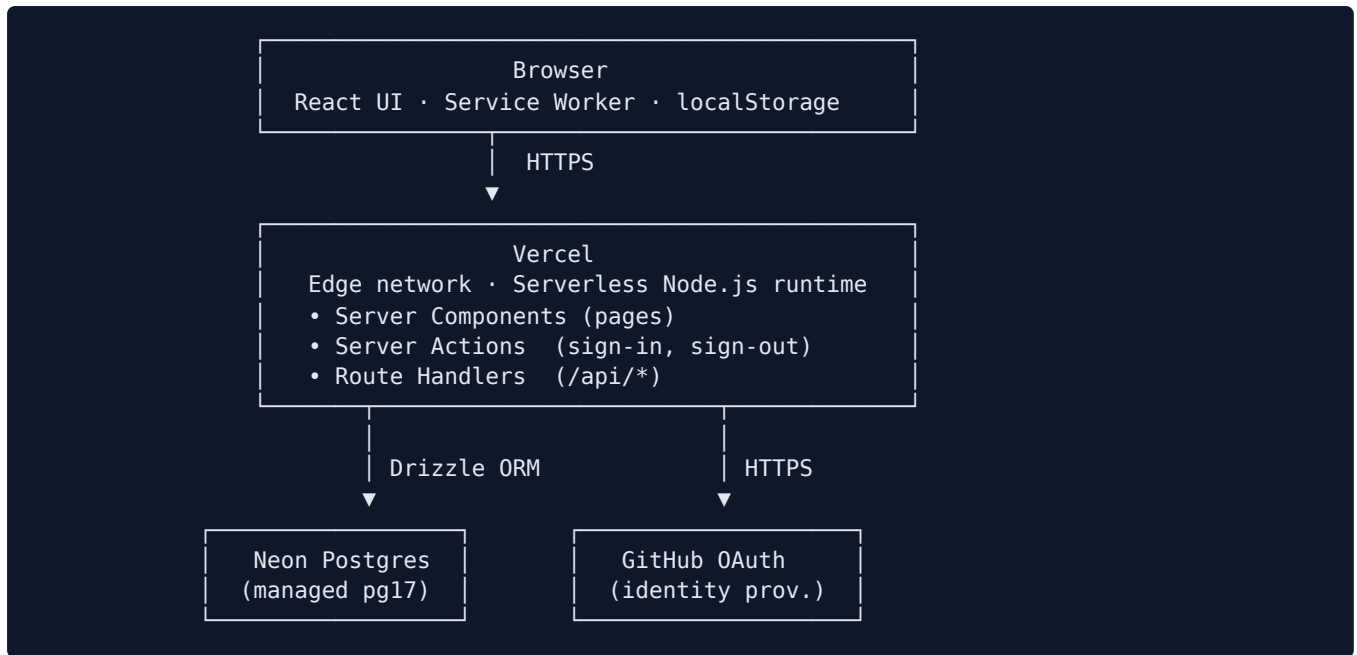
- HTTP basics (request, response, status codes, headers, cookies).
- TypeScript / JavaScript fundamentals.
- What a relational database is and roughly how SQL works.
- Git basics.

You don't need to know:

- Any particular framework. We'll explain Next.js as we go.
- The Transformer model. The maths is the *content* of the site, not the subject of this report.

Throughout this document, file paths are written like `src/lib/auth/config.ts:97-110` so you can open the actual code while reading. The repository is at github.com/BrendanJamesLynskey/transformer-explainer.

1. The shape of the system, in one diagram



The **only** stateful pieces are the Postgres database and the user's browser cookies. Everything between them is functions: the same code that runs locally on `pnpm dev` runs on Vercel as serverless invocations, spun up per-request.

This is the **stateless server** pattern. Each HTTP request lands on a fresh function instance (or a recently-warmed one), reads what it needs from Postgres, computes a response, and exits. There is no in-memory state that survives across requests except for tiny short-lived caches (rate-limit buckets, the lazy-loaded sanitiser).

2. The technology stack — what each piece is for

Layer	Choice	What it is
Runtime	Node.js 24	The JavaScript engine the server code runs on.
Framework	Next.js 14 (App Router)	A meta-framework on top of React. Decides what runs server-side vs client-side, handles routing, builds pages.
Language	TypeScript (strict)	JavaScript with static types. The <code>strict</code> flag turns on every safety check.
ORM	Drizzle	A <i>thin</i> TypeScript wrapper that lets us write SQL-like code that's type-checked.
Database	Postgres (Neon)	The relational database. Neon is a managed Postgres provider with a free tier.
Auth	Auth.js v5 (NextAuth)	A library that handles OAuth flows, session cookies, and account linking. Talks to Postgres via Drizzle.
Validation	Zod	Runtime schema validation. We use it on every API input.
Styling	Tailwind CSS	A CSS framework where you compose styles by adding utility class names directly to JSX elements.
Visuals	D3.js	A library for binding data to DOM elements (charts, heatmaps).
Content	MDX (via <code>next-mdx-remote/rsc</code>)	Markdown with React components inline. Each tutorial chapter is one MDX file.
Tests	Vitest + Playwright	Vitest for unit tests; Playwright for end-to-end browser tests.
Lint/format	ESLint + Prettier	Lint flags suspicious code; Prettier formats it.
Deploy	Vercel	A cloud platform that builds the app from Git and hosts the serverless functions.
CI	GitHub Actions	Runs lint, typecheck, tests, and a math-correctness script on every PR.

When choosing a stack for a real job you'd weigh team familiarity, hiring pool, ecosystem, and operational cost. For this project the constraint was: **nothing exotic** — every piece must be something a junior engineer will plausibly meet at a normal employer.

3. What happens when you visit `/learn/03-attention`

Let's trace one request from your browser address bar all the way to the HTML that paints on screen. This is the most important paragraph in the report — every other section is detail expanding on this trace.



3.1 DNS and the edge

You type `transformer-explainer-three.vercel.app/learn/03-attention` and hit Enter.

1. Your operating system asks DNS for the IP address of that host.
2. DNS returns an IP belonging to **Vercel's edge network** — a fleet of small servers placed near your physical location.
3. Your browser opens an HTTPS (TLS) connection to that IP.
4. The edge server receives `GET /learn/03-attention`. It looks up the project's routing table. The path matches the dynamic route `app/learn/[slug]/page.tsx`, so the edge **invokes the corresponding serverless function**.

3.2 The serverless function spins up

If no instance of this function is currently warm in your nearest region (typically `iad1` or `lhr1`), Vercel cold-starts one: a Node.js process is created, our compiled code is loaded, and execution begins. Cold starts take ~200-500 ms.

If a warm instance exists, the request goes straight to it — typically under 30 ms. Vercel's "Fluid Compute" lets one warm instance handle many concurrent requests (it's not strict one-request-per-instance).

3.3 The Server Component renders

The route handler is `src/app/learn/[slug]/page.tsx`. Looking at it:

```
export default async function SectionPage({
  params,
}): {
  params: { slug: string };
}: Promise<JSX.Element> {
  if (!isValidSlug(params.slug)) notFound();
  const meta = getSectionMeta(params.slug);
  const mdx = await readSectionMdx(params.slug);
  if (mdx === null) notFound();

  // ... look up prev/next sections in the catalogue ...

  const session = await runOrFallback("learn:auth", getSession, null);
  const userId = (session?.user as { id?: string } | undefined)?.id ?? null;

  return (
    <article>
      { /* JSX rendering MDX, comments, progress tracker, etc. */ }
    </article>
  );
}
```

This is a **React Server Component** (RSC). It's a function that runs on the server, returns JSX, and never ships to the browser. Notice it can `await` directly — no `useEffect`, no `useState`. Server Components are just plain async functions.

The JSX it returns is **serialised** by Next.js into HTML and sent to the browser. Anything that needs interactivity (button clicks, form state) is done by separate **Client Components** marked with `"use client"` — those *do* ship to the browser as JavaScript.

3.4 Reading the MDX from disk

`readSectionMdx("03-attention")` is in `src/lib/mdx/sections.ts`. It opens `content/decoder/03-attention.mdx` from the bundled assets and reads it as a string. MDX is Markdown plus inline JSX components:

```
# Attention

<Layer kind="concept">

  **Attention** is how the decoder looks back at earlier tokens...

</Layer>

<AttentionWidget />
```

`<MDXRemote source={mdx} components={mdxComponents} />` later in the page parses this string, swaps `<Layer>` and `<AttentionWidget>` for actual React components from `src/lib/mdx/components.ts`, and renders the result. The `mdxComponents` map is the only API our content authors need: add a new kind of widget to the map and any `.mdx` file can use it.

3.5 Looking up the user's session

`getSession()` is a thin wrapper around Auth.js's `auth()`. It does roughly:

1. Read the `__Secure-next-auth.session-token` cookie from the request.

2. Look that token up in the `sessions` table (production strategy).
3. Join to `users` to get the row.
4. Return the user object, or `null` if no session.

If the cookie is missing or expired, `null` is returned. The page renders fine without it — the comments form just shows a "Sign in to leave a comment" hint instead of a textarea.

In production this query takes ~5 ms against Neon.

3.6 The `runOrFallback` wrapper

You'll see this pattern everywhere:

```
const session = await runOrFallback("learn:auth", getSession, null);
```

It's defined in `src/lib/db-fallback.ts`:

```
export async function runOrFallback<T>(  
  key: string,  
  fn: () => Promise<T>,  
  fallback: T,  
) : Promise<T> {  
  try {  
    return await fn();  
  } catch (err) {  
    maybeWarn(key, err);  
    return fallback;  
  }  
}
```

If the database is unreachable (placeholder env values, DB outage, network blip), the page **still renders** — it just acts as if the user isn't signed in. The throttled `console.warn` (max one per minute per call site) means our log volume stays sane even in a sustained outage.

This is one of the most important practical lessons in backend engineering: **degrade gracefully**. A page that paints without comments is enormously better than a 500 error.

3.7 Server Components compose

Your page returns more JSX, including `<EventTracker sectionSlug={...}/>` and `<CommentSection ...>`. These are Client Components. Next.js serialises the whole tree to HTML, then attaches a sidebar payload telling the browser "here's the data those Client Components need to hydrate themselves on first paint".

The browser receives:

```
HTTP/2 200 OK  
content-type: text/html; charset=utf-8  
strict-transport-security: max-age=63072000; includeSubDomains; preload  
x-vercel-cache: MISS  
  
<!DOCTYPE html>  
...
```

It paints the page, then runs a small amount of JS to **hydrate** the Client Components — re-attaching event listeners, replaying any state the server set up, and starting things like the EventTracker beacon.

3.8 The Client Components fire their own requests

`<EventTracker sectionSlug="03-attention" />` is a "use client" component. When it mounts in the browser, its `useEffect` fires:

```
await fetch("/api/events", {
  method: "POST",
  headers: { "content-type": "application/json" },
  body: JSON.stringify({
    sessionId,
    events: [{ kind: "page_view", sectionSlug }],
  }),
  keepalive: true,
});
```

This is a fresh HTTP request to the same Vercel deployment, hitting our `/api/events` Route Handler. We'll trace that one in §6.

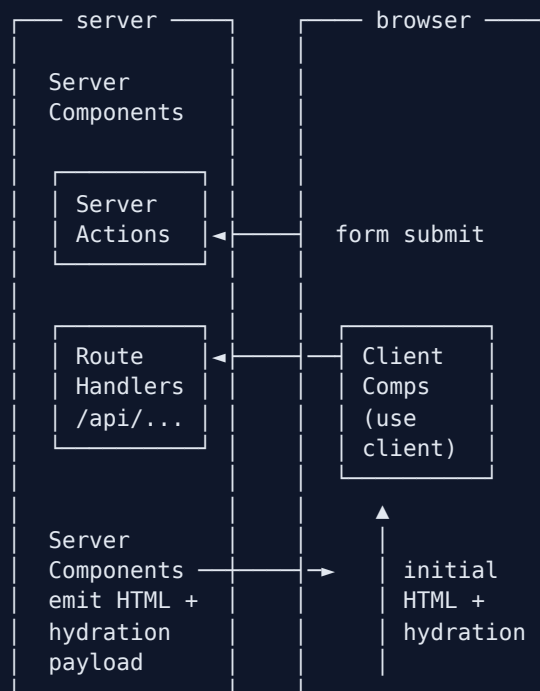
So one user visit to `/learn/03-attention` produces:

1. One `GET /learn/03-attention` to render the page.
2. Several `POST /api/compute/*` requests as the `<AttentionWidget>` computes Q/K/V matrices from the typed input.
3. A `GET /api/sections/03-attention/comments` to populate the comments list.
4. A `POST /api/events` from the `EventTracker`.
5. (If signed in) a `POST /api/progress` from the `ProgressTracker`.

Each of these is a separate, stateless function invocation on Vercel.

4. The Next.js mental model

Coming from a more traditional backend (Django, Rails, Express + a separate React frontend) it takes a moment to get this straight. There are exactly **four** kinds of code in our `src/` tree:



The diagram shows where each kind of code lives. Server Components and Server Actions only ever run on the server (no JS shipped). Client Components run on both: server-side to produce initial HTML, then client-side to hydrate.

Server Components (default)

`.tsx` files in `src/app/**/page.tsx`, `layout.tsx`, etc. They run **only on the server**. They can be `async`, can read files, can talk to the database directly. Their output is HTML.

```
// src/app/learn/page.tsx
export default async function LearnIndex(): Promise<JSX.Element> {
  const status = await Promise.all(SECTIONS.map(...)); // disk read
  const session = await runOrFallback("learn-index:auth", getSession, null);
  const progress = await listForUser(userId); // DB read
  return <main>...</main>;
}
```

You will never see `useState` or `useEffect` in a Server Component because **there is no client to handle state**. Server Components are re-executed top to bottom on every request (well, on every cache miss — Next.js does aggressive caching, but our pages have `dynamic = "force-dynamic"` to opt out).

Client Components ("use client")

`.tsx` files starting with `"use client";`. They run on the server *for the initial HTML* and then **also in the browser**, where they hydrate into interactive React. This is where `useState`, event handlers, `fetch()` calls happen.


```
// src/components/interactive/EventTracker.tsx
"use client";

import { useEffect } from "react";

export function EventTracker({ sectionSlug }) {
  useEffect(() => {
    void postEvent(sessionId, "page_view", sectionSlug);
    // ...
  }, [sectionSlug]);
  return null;
}
```

Anything imported from this file becomes part of the JavaScript bundle the browser downloads. Keep them small.

Server Actions

Functions inside a Server Component file that begin with `"use server"`. When the form is submitted, the browser sends a special HTTP POST that Next.js routes to that exact function. No API endpoint to define, no JSON to serialise.

```
// src/components/ui/SiteHeader.tsx
async function signInAction() {
  "use server";
  await signIn("github", { redirectTo: "/" });
}

export async function SiteHeader() {
  return <form action={signInAction}>
    <button type="submit">Sign in with GitHub</button>
  </form>;
}
```

Behind the scenes, Next.js generates a hidden form input with an action ID and an internal route. You won't see this in our `/api/` directory.

Route Handlers (`/api/*/route.ts`)

Plain HTTP endpoints. The file's exported `GET`, `POST`, `PATCH`, `DELETE` functions become the handler for the corresponding HTTP method.

```
// src/app/api/events/route.ts
export const runtime = "nodejs";
export const dynamic = "force-dynamic";

export async function POST(req: Request): Promise<Response> {
  // ...validate body, look up session, insert events...
  return NextResponse.json({ ok: true, data: { written } });
}
```

These look like Express handlers — closest thing to "traditional" backend code in the project.

The choice between **Server Action** and **Route Handler** comes down to: *who calls it?* Server Actions are designed to be invoked from forms inside our own pages. Route Handlers are designed to be called by anyone — JS clients, mobile apps, `curl`. Anything that the browser needs to call directly via `fetch()` should be a Route Handler.

5. The database layer

5.1 Connection management

`src/lib/db/client.ts` exports a single `db` object. Every part of the app imports from this one place:

```
import { db } from "@lib/db/client";
import { eq } from "drizzle-orm";
import { users } from "@lib/db/schema";

const rows = await db.select().from(users).where(eq(users.id, id));
```

The implementation is hot-reload-safe:

```
const globalForPg = globalThis as unknown as {
  pgPool?: ReturnType<typeof postgres>;
};

function makePool() {
  if (!env.DATABASE_URL) throw new Error("DATABASE_URL is not set...");
  return postgres(env.DATABASE_URL, {
    ssl: needsSsl(env.DATABASE_URL) ? "require" : false,
    max: 10,
  });
}

const pool = globalForPg.pgPool ?? makePool();
if (process.env.NODE_ENV !== "production") globalForPg.pgPool = pool;

export const db = drizzle(pool, { schema });
```

The `globalThis` trick is important. In dev, Next.js hot-reloads modules on every file save — without this guard, every save would create a new Postgres connection pool, leaking sockets until the dev server crashes. By stashing the pool on `globalThis` (which survives reloads) we reuse one pool. In production we don't bother because the process gets a fresh runtime per cold start.

5.2 Why a connection pool

A database connection isn't free. Each one is a TCP socket plus backend memory on the Postgres server. Establishing one round-trips TCP, TLS, and authentication — ~10-50 ms.

A *pool* keeps a bunch of connections open and hands them out as needed. Our `max: 10` means up to 10 concurrent queries in flight from one serverless instance.

5.3 The schema

`src/lib/db/schema.ts` declares the tables in a TypeScript DSL:

```

export const users = pgTable("users", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name"),
  email: text("email").unique(),
  emailVerified: timestamp("email_verified", { withTimezone: true }),
  image: text("image"),
  githubId: text("github_id"),
  githubLogin: text("github_login"),
});

export const comments = pgTable("comments", {
  id: uuid("id").defaultRandom().primaryKey(),
  sectionSlug: text("section_slug").notNull(),
  userId: uuid("user_id")
    .notNull()
    .references(() => users.id, { onDelete: "cascade" }),
  parentId: uuid("parent_id"),
  bodyMd: text("body_md").notNull(),
  hidden: boolean("hidden").notNull().default(false),
  createdAt: timestamp("created_at", { withTimezone: true })
    .notNull()
    .default(sql`now()`),
  updatedAt: timestamp("updated_at", { withTimezone: true })
    .notNull()
    .default(sql`now()`),
});

```

The relationships between the eight tables look like this:



A few things to notice:

- **uuid with defaultRandom()** . We use UUIDs for primary keys, not serial integers. This means we can generate IDs client-side, merge rows from different shards, and never expose a row count to the internet. Postgres has built-in `gen_random_uuid()` for this.
- **references() => users.id, { onDelete: "cascade" }** . A foreign key constraint. If a user row is deleted, all their comments delete too. Postgres enforces this — *you cannot* leave dangling rows.
- **withTimezone: true on timestamps**. Postgres stores these in UTC and returns them with timezone info. Always do this; "naïve" timestamps are a perennial source of off-by-an-hour bugs.
- **.NotNull() on every required column**. Defaults to nullable in SQL, but we make nullability the explicit, deliberate choice.

5.4 Migrations

When we change the schema, Drizzle generates a SQL migration file:

```

pnpm db:generate    # creates a new file in drizzle/
pnpm db:migrate     # applies it to the configured DB

```

Each migration is a plain `.sql` file with timestamps, committed to git. That's the **single source of truth for production schema state**. Never modify a migration after it's been applied; instead, write a new one that fixes whatever was wrong.

For this project we use `pnpm db:push` (Drizzle's "just diff and apply" mode) because we're a single-developer, demo-grade project. In a team setting you would absolutely use migrations, because they let you review and reproduce schema changes deterministically.

5.5 Querying

Drizzle queries look like SQL written in TypeScript:

```
// Read all top-level comments for a section
const rows = await db
  .select()
  .from(comments)
  .where(and(eq(comments.sectionSlug, slug), isNull(comments.parentId)))
  .orderBy(asc(comments.createdAt));

// Insert with returning
const inserted = await db
  .insert(comments)
  .values({ sectionSlug, userId, parentId: null, bodyMd: body })
  .returning();
```

The killer feature: the result is **fully typed**. `inserted[0].bodyMd` is `string`, `inserted[0].hidden` is `boolean`, autocomplete works, and typos are compile errors. There is no runtime type validation on query results because the types come from the schema declaration directly.

5.6 The monotonic upsert — a real-world pattern

`src/lib/progress.ts:47-69` has a quietly clever bit of SQL:

```
await db
  .insert(progress)
  .values({ userId, sectionSlug, status })
  .onConflictDoUpdate({
    target: [progress.userId, progress.sectionSlug],
    set: {
      status: sql`CASE
        WHEN ${progress.status} = 'completed' THEN ${progress.status}
        WHEN ${progress.status} = 'in_progress' AND ${newRank} >= 1 THEN ${status}
        WHEN ${progress.status} = 'not_started' THEN ${status}
        ELSE ${progress.status}
      END`,
      updatedAt: new Date(),
    },
  });
```

The problem this solves: on every page visit, the ProgressTracker fires `status: "in_progress"` on mount, then escalates to `"completed"` once the user scrolls past 80% with a click. But on a *return* visit to a section the user already completed, the tracker re-fires `in_progress`.

If the upsert just blindly overwrote, the user's progress would *regress* from `completed` to `in_progress`. That's wrong.

The `CASE` expression encodes the state machine: only allow forward transitions. If the row already says `completed`, ignore the new value. If it says `in_progress`, accept anything $\geq$ `in_progress`. If it says `not_started`, accept

anything.

This kind of integrity logic in SQL is faster, race-free, and clearer than reading-then-writing in application code. The Postgres engine handles the concurrent-update case for free.

6. API design — three flavours of route

Our `/api/*` directory has three distinct categories of route:

6.1 Pure compute routes (no auth, no DB)

`/api/compute/{embed,attention,ffn,forward,sample}` execute pure-TypeScript transformer maths. Their handler looks like:

```
// pseudocode for illustration
export async function POST(req: Request) {
  const body = await req.json();
  const parsed = inputSchema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json(
      { ok: false, error: parsed.error.issues[0].message },
      { status: 400 },
    );
  }

  const result = embed(parsed.data.text, parsed.data.seed);
  return NextResponse.json({ ok: true, data: result });
}
```

CPU-bound, no I/O. These run fast (~5-30 ms) on a warm function. They're the safest endpoints in the system because they touch no shared state.

6.2 Auth-and-DB routes

`/api/sections/[slug]/comments`, `/api/experiments`, `/api/progress`, `/api/admin/metrics`, `/api/comments/[id]`. These do real work:

1. Parse the request.
2. Look up the session.
3. Authorise (am I signed in? am I the owner? am I admin?).
4. Validate input with Zod.
5. Do a SQL query.
6. Return JSON.

A representative example, `src/app/api/sections/[slug]/comments/route.ts`:

```

export async function POST(req, ctx) {
  const session = await auth();
  const userId = (session?.user as { id?: string })?.id;
  if (!userId) {
    return NextResponse.json({ ok: false, error: "Sign in." }, { status: 401 });
  }
  if (!isValidSlug(ctx.params.slug)) {
    return NextResponse.json({ ok: false, error: "Unknown section" }, { status: 404 });
  }

  const body = await req.json();
  const parsed = createCommentSchema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json(
      { ok: false, error: parsed.error.issues[0]?.message ?? "Invalid input" },
      { status: 400 },
    );
  }
  if (await isReplyOfReply(parsed.data.parentId)) {
    return NextResponse.json(
      { ok: false, error: "Threading is one level deep – reply to the top-level comment." },
      { status: 400 },
    );
  }

  const created = await create(userId, ctx.params.slug, parsed.data);
  return NextResponse.json(
    { ok: true, data: { id: created.id, /* ... */ } },
    { status: 201 },
  );
}

```

Notice the **uniform response shape**: `{ ok: true, data }` or `{ ok: false, error: string }`. Every endpoint in the project follows this. Clients always know which key to look at — no surprise null pointers.

6.3 Analytics ingestion (anon-friendly)

`/api/events` is special: it accepts data from signed-out visitors too. It uses a stable per-tab UUID stored in `localStorage` to tie a visitor's events together without authentication.

```
export async function POST(req: Request) {
  const ip = req.headers.get("x-forwarded-for")?.split(",")[0]?.trim()
  ?? req.headers.get("x-real-ip")
  ?? "anon";
  if (!rateLimitOk(`events:${ip}`, 60, 1)) {
    return NextResponse.json({ ok: false, error: "Slow down." }, { status: 429 });
  }
  const parsed = ingestSchema.safeParse(await req.json());
  if (!parsed.success) {
    return NextResponse.json(
      { ok: false, error: parsed.error.issues[0]?.message },
      { status: 400 },
    );
  }
  const session = await runOrFallback("events:auth", () => auth(), null);
  const userId = (session?.user as { id?: string } | undefined)?.id ?? null;
  const written = await runOrFallback(
    "events:insert",
    () => recordEvents(userId, parsed.data),
    0,
  );
  return NextResponse.json({ ok: true, data: { written } });
}
```

Three things are happening here that you don't see in the auth+DB routes:

- **Rate limiting.** A simple in-memory token bucket (more on this in §11). Returns 429 if a single IP fires more than 60 events per minute.
- **Auth lookup is wrapped in `runOrFallback`**. Even if our session store is unavailable, anonymous events still go through. Analytics data should never block the user experience.
- **The DB insert is also wrapped.** If Postgres is down, we still return `ok: true` to the client. We've decided analytics-loss is preferable to user-visible errors.

This is a pragmatic choice. For *important* writes (sign up, save a saved-experiment) we wouldn't do this — we'd surface the error.

6.4 Why every input goes through Zod

Zod is a runtime schema validator. It looks like this:

```
export const ingestSchema = z.object({
  sessionId: z.string().min(8).max(64),
  events: z
    .array(z.object({
      kind: z.enum(EVENT_KIND),
      sectionSlug: z.string().min(1).max(80).optional(),
      meta: z.record(z.unknown()).optional(),
    }))
    .min(1)
    .max(50),
});

const parsed = ingestSchema.safeParse(body);
if (!parsed.success) { /* return 400 */ }
```

Zod gives us three things at once:

1. **Sanitised data.** After `safeParse`, `parsed.data` is exactly the shape we declared. No extra fields, no missing required fields, no wrong types. No need to defensively check `if (typeof body.x === "string")`

everywhere.

2. **TypeScript inference.** `type Ingest = z.infer<typeof ingestSchema>` gives us the static type for free.
3. **Cheap denial-of-service protection.** Limits like `.array(...).max(50)` mean an attacker cannot send a 10-million-element array and force the server to spend forever validating.

You should validate **every** untrusted input. The only "untrusted" data source is the network. Anything from your own database or your own files is trusted (you put it there).

7. Authentication and sessions

The auth surface is one of the most error-prone parts of any web app. We use Auth.js v5, which handles the OAuth dance, CSRF tokens, and session cookies. Our job is to plug in providers and configure callbacks.

7.1 The OAuth flow



When you click "Sign in with GitHub":

1. **Browser** → server action `signInAction` → `signIn("github", ...)`.
2. Auth.js generates a CSRF token, sets a `__Host-next-auth.csrf-token` cookie, and returns a redirect to:

```
https://github.com/login/oauth/authorize
?client_id=0v23lia1...
&scope=read:user user:email
&state=<random>
&redirect_uri=https://transformer-explainer-three.vercel.app/api/auth/callback/github
```

3. **Browser** follows the redirect to GitHub.
4. GitHub asks the user to consent (or shows nothing if already consented previously).
5. GitHub redirects the browser back to the `redirect_uri` with `?code=<short-lived-code>&state=<echoed-back>`.
6. Auth.js's callback handler at `/api/auth/callback/github`:
 - Verifies the `state` parameter matches the cookie (CSRF).
 - Exchanges the `code` for an access token (server-to-server POST).
 - Calls GitHub's `/user` endpoint to fetch the profile.
 - Calls our `mapGitHubProfile` to map the profile into our user shape.
 - **Inserts** rows into `users` and `accounts` if the user is new, or **links** the GitHub identity to an existing matching user.
 - **Inserts** a row into `sessions` with a fresh session token.
 - Sets the `__Secure-next-auth.session-token` cookie on the response.
 - Redirects to the original `redirectTo` (default: `/`).
7. **Browser** lands on `/` with the session cookie. Subsequent requests carry the cookie, and our `auth()` calls return a populated session.

7.2 Session strategies — database vs JWT

— Database strategy (production) —

Browser cookie holds opaque ID	Server auth() reads cookie reads sessions row returns user	Postgres sessions row
logout: delete cookie server-side revoke: just delete the row → instant	auth() returns null	

— JWT strategy (E2E test build) —

Browser cookie IS the session (signed JWT containing user info)	Server auth() decrypts cookie returns user from claims	Postgres not used for reads
logout: delete cookie	pre-existing JWTs valid until exp; can't revoke without DB lookup	

Auth.js supports two modes:

- **Database sessions** (production). The session cookie holds an opaque token; the actual session data lives in the `sessions` table. Logging out is just deleting the row. Revocation is instantaneous.
- **JWT sessions** (E2E test build). The cookie holds a signed JWT containing the user info. No DB lookup per request. Logging out only deletes the cookie — old JWTs remain valid until expiry.

Our config flips the strategy based on `E2E_TEST_AUTH`:

```
session: { strategy: e2eAuthEnabled ? "jwt" : "database" },
```

JWT mode is required because the `Credentials` provider (which we use for E2E sign-in with a fake username) only supports JWT. Production keeps database sessions for stronger logout guarantees.

7.3 The `mapGitHubProfile` UUID gotcha

Here's a story from this project. Originally `mapGitHubProfile` returned:

```
return {
  id: String(profile.id),
  name: profile.name ?? profile.login,
  // ...
};
```

GitHub's `profile.id` is a numeric user ID like `12345`. `Auth.js` passes this object to our Drizzle adapter, which calls `INSERT INTO users (id, ...) VALUES ('12345', ...)`. But our `users.id` column is type `uuid`. Postgres rejects it: *invalid input syntax for type uuid*.

The symptom in production was `OAuthCallbackError` with no obvious cause until we read the runtime logs. The fix was to *not* return `id` from the mapper:

```
return {
  // No `id` – let Postgres generate a real UUID via defaultRandom().
  name: profile.name ?? profile.login,
  email: profile.email ?? null,
  image: profile.avatar_url ?? null,
  githubId: String(profile.id),
  githubLogin: profile.login,
};
```

This is a textbook example of why **types alone aren't enough**. The TypeScript compiler was happy with `id: string`, but the database demanded `id: uuid`. You have to know your storage layer.

7.4 The `OAuthAccountNotLinked` story

When `db:seed` ran, it inserted a placeholder admin user with `email: "brendanlynskey@googlemail.com"` and `githubId: "seed:placeholder"`. On real first sign-in via GitHub, the email matched but the GitHub account didn't link to anything, so `Auth.js` refused with `OAuthAccountNotLinked` — a security feature against email-collision account takeover.

The fix is one config flag on the GitHub provider:

```
const githubProvider = GitHub({
  clientId: process.env.AUTH_GITHUB_ID,
  clientSecret: process.env.AUTH_GITHUB_SECRET,
  profile: mapGitHubProfile,
  allowDangerousEmailAccountLinking: true,
});
```

This says: if a `users` row with the same email already exists, link the GitHub identity to it. Safe specifically because GitHub never lets a user claim an email they haven't verified — a hostile party can't register a github.com user with someone else's verified email.

Without this flag the safer behaviour is the default, but for our seed-then-real-OAuth scenario it's the right call. Generally, you disable it for any provider whose email-verification you don't fully trust.

7.5 The redirect-loop saga

Our config originally had:

```
pages: { signIn: "/api/auth/signin" },
```

The intent was "use the default Auth.js sign-in page". But in v5, `pages.signIn` should point at a *custom UI page you own*, never at the Auth.js handler itself. Setting it to the handler means Auth.js, when trying to redirect to "the sign-in page" as a fallback, redirects to its own handler — which redirects again — `ERR_T00_MANY_REDIRECTS`.

Fix: remove the field entirely. Auth.js's built-in sign-in handler at `/api/auth/signin` auto-redirects to the only configured provider when there's just one.

These three bugs (the UUID, the email linking, the redirect loop) all hit only in production. They're great illustrations of why you need **runtime logs** in production and how to read them.

8. Comments — Markdown rendering and XSS

Users post Markdown. We render HTML. If we just dropped their input into the page we'd have a textbook XSS vulnerability — they could send `<script>` tags or event handlers that execute in other users' browsers.

`src/lib/comments-render.ts` has the full pipeline:

```
export function renderCommentHtml(md: string): string {
  const rawHtml = marked.parse(md, { async: false }) as string;
  return getSanitize()(rawHtml, {
    USE_PROFILES: { html: true },
    FORBID_TAGS: ["style", "script"],
    FORBID_ATTR: ["style", "onerror", "onload", "onclick"],
  });
}
```

Two stages:

1. **marked** parses Markdown into HTML.
2. **DOMPurify** (via `isomorphic-dompurify`) walks the resulting HTML tree and removes anything dangerous: `<script>`, `<iframe>`, inline event handlers like `onclick=`, `javascript:` URLs in links, etc.

Our test file `tests/unit/comments.test.ts` actually probes for each attack class:

```
it("strips raw <script> tags", () => {
  const html = renderCommentHtml("Hi <script>alert('xss')</script> there");
  expect(html).not.toMatch(/<script/i);
  expect(html).toMatch(/Hi/);
});

it("strips inline event handler attributes from raw HTML", () => {
  const html = renderCommentHtml('');
  expect(html).not.toMatch(/onerror=/i);
});

it("blocks javascript: URLs in links", () => {
  const html = renderCommentHtml("[bad](javascript:alert(1))");
  expect(html).not.toMatch(/javascript:alert/i);
});
```

A subtle production issue we hit: `isomorphic-dompurify` initialises JSDOM at *import* time. JSDOM tries to read a default stylesheet from disk — and during Vercel's build-time route data collection that file isn't available, so the build crashed.

The fix was a **lazy import**:

```
let sanitizeCache: SanitizeFn | null = null;
function getSanitize(): SanitizeFn {
  if (sanitizeCache) return sanitizeCache;
  // eslint-disable-next-line @typescript-eslint/no-require-imports
  const mod = require("isomorphic-dompurify");
  sanitizeCache = mod.sanitize.bind(mod);
  return sanitizeCache;
}
```

JSDOM only initialises on the first call, which is at *request* time, not build time. The cached function pointer means subsequent calls are free. This is a practical instance of the **lazy initialisation** pattern.

The lesson: a "pure" library can have surprising side-effects in its top-level module code. Always import expensive things lazily if your test or build environment differs from your runtime.

9. Analytics — anonymous DAU and a homemade rate limiter

`src/lib/analytics.ts` has the read side of the analytics:

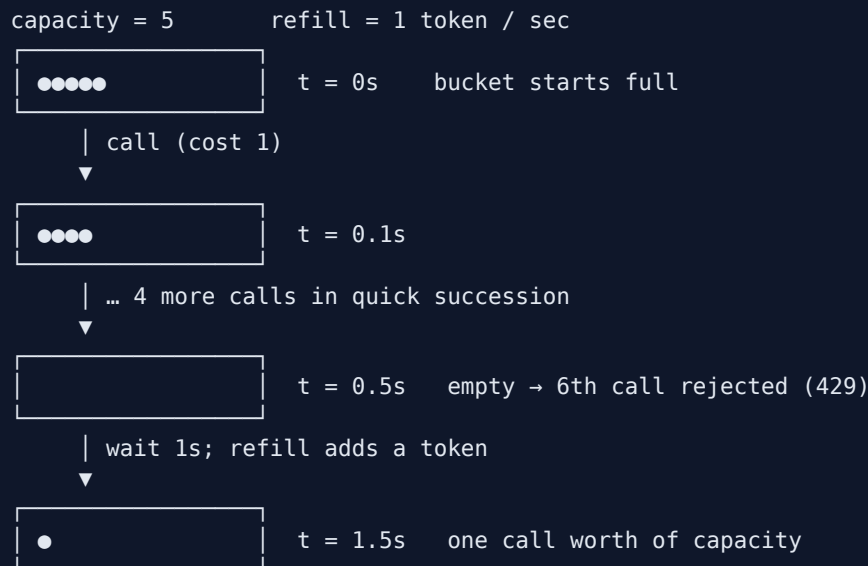
```
export async function dailyActive(days: number): Promise<DailyActive[]> {
  const since = new Date(Date.now() - days * 24 * 3600 * 1000);
  const rows = await db
    .select({
      day: sql<string>`to_char(date_trunc('day', ${events.createdAt})) at time zone 'UTC', 'YYYY-MM-DD'`,
      users: sql<number>`count(distinct ${events.sessionId})::int`,
    })
    .from(events)
    .where(gte(events.createdAt, since))
    .groupBy(sql`date_trunc('day', ${events.createdAt}) at time zone 'UTC'`)
    .orderBy(sql`date_trunc('day', ${events.createdAt}) at time zone 'UTC'`);
  return rows;
}
```

A few things to digest:

- `sql<string>\ ...`` is Drizzle's escape hatch for raw SQL when you need expressions the DSL doesn't model. The `<string>` is a type hint so the result is correctly typed.
- `date_trunc`. A Postgres function that floors a timestamp to a given precision. `date_trunc('day', ts)` gives midnight UTC of `ts`'s day.
- `count(distinct ${events.sessionId})::int`. The `::int` is a Postgres cast — `count(distinct ...)` returns `bigint` by default, which arrives as `string` in JavaScript (because JS numbers can't represent every `bigint`). For a daily active count that's never going to exceed `Number.MAX_SAFE_INTEGER`, casting to `int` is fine.

The `sessionId` is a v4 UUID generated client-side and stored in `localStorage`. We use it because most events are anonymous — we wouldn't have a `userId` to count. As long as the same browser tab is reused, its events tie together.

9.1 The token-bucket rate limiter



src/lib/analytics-shared.ts:50-69 :

```
const BUCKETS = new Map<string, { tokens: number; lastRefill: number }>();

export function rateLimitOk(
  key: string,
  capacity: number,
  refillPerSec: number,
  now: number = Date.now(),
): boolean {
  const b = BUCKETS.get(key) ?? { tokens: capacity, lastRefill: now };
  const dt = (now - b.lastRefill) / 1000;
  b.tokens = Math.min(capacity, b.tokens + dt * refillPerSec);
  b.lastRefill = now;
  if (b.tokens < 1) {
    BUCKETS.set(key, b);
    return false;
  }
  b.tokens -= 1;
  BUCKETS.set(key, b);
  return true;
}
```

This is the **token bucket** algorithm:

- Each unique `key` (here, `events:<ip>`) has a bucket holding up to `capacity` tokens.
- Tokens refill at `refillPerSec` per second, up to `capacity`.
- Each call costs one token. If the bucket is empty, the call is rejected.

It's a classic algorithm worth knowing. The implementation is a few lines, tested in `tests/unit/analytics.test.ts`:

```

it("allows up to capacity then rejects", () => {
  const now = 1_000_000;
  expect(rateLimitOk("k", 3, 1, now)).toBe(true);
  expect(rateLimitOk("k", 3, 1, now)).toBe(true);
  expect(rateLimitOk("k", 3, 1, now)).toBe(true);
  expect(rateLimitOk("k", 3, 1, now)).toBe(false);
});

it("refills over time", () => {
  const t0 = 1_000_000;
  expect(rateLimitOk("k", 1, 1, t0)).toBe(true);
  expect(rateLimitOk("k", 1, 1, t0)).toBe(false);
  expect(rateLimitOk("k", 1, 1, t0 + 1500)).toBe(true);
});

```

The key idea: `now` is a parameter, not `Date.now()` directly. That makes the function deterministic and testable. In production we let it default to the real clock; in tests we pass made-up timestamps.

Caveat: this is in-memory per serverless instance. With three warm instances running, an attacker's effective rate limit triples. For a proper rate limiter at scale you'd use Redis, Upstash, or Vercel Edge Config. For our use case (a portfolio site), it's enough — and the fact that we're aware of the limitation matters more than fixing it.

10. The environment-variables boundary

`src/lib/env.ts` is the **only** file that reads `process.env`. Every other piece of code imports the typed `env` object:

```

import { env } from "@lib/env";
const url = env.DATABASE_URL;

```

Why? Because this is the single chokepoint where we validate the inputs to the system at boot. If a deploy is misconfigured, the app crashes at startup with a clear message instead of with a confusing 500 on the first request that happens to need that variable.

The implementation uses Zod:


```

export function makeEnvSchema(isProd: boolean) {
  const requiredInProd = (label: string) =>
    z.string().min(1).or(z.literal("")).transform(() => undefined).optional()
    .superRefine((val, ctx) => {
      if (isProd && !val) {
        ctx.addIssue({
          code: z.ZodIssueCode.custom,
          message: `${label} is required in production`,
        });
      }
    });

  return z.object({
    NODE_ENV: z.enum(["development", "test", "production"]).default("development"),
    DATABASE_URL: z.string().url()
      .refine((s) => s.startsWith("postgres://") || s.startsWith("postgresql://"), {
        message: "DATABASE_URL must be a postgres:// or postgresql:// URL",
      })
      .optional()
      .superRefine(/* required in prod */),
    AUTH_SECRET: requiredInProd("AUTH_SECRET"),
    AUTH_GITHUB_ID: requiredInProd("AUTH_GITHUB_ID"),
    AUTH_GITHUB_SECRET: requiredInProd("AUTH_GITHUB_SECRET"),
    NEXT_PUBLIC_SITE_URL: z.string().url().default("http://localhost:3000"),
    ADMIN_GITHUB_LOGINS: z.string().default(""),
    MAX_SEQ_LEN: z.coerce.number().int().positive().max(64).default(16),
    MAX_D_MODEL: z.coerce.number().int().positive().max(256).default(64),
    MAX_BLOCKS: z.coerce.number().int().positive().max(8).default(4),
    NEXT_PUBLIC_DEV_DEMOS_ENABLED: z.enum(["true", "false"]).default("false")
      .transform((v) => v === "true"),
  });
}

```

A few things to note:

- **Two-mode validation.** Required-in-prod only enforces in production builds. In dev with placeholder values, the app still boots, just without the auth/DB features.
- **.coerce.number()** . Env vars are always strings; **.coerce** converts before validation.
- **NEXT_PUBLIC_*** . Next.js inlines these into the browser bundle. Anything else stays server-side. Never put a secret in a **NEXT_PUBLIC_*** variable — the user can read it.

When the validation fails, we print every issue and **throw** . The error in deploy logs becomes:

```

❌ Invalid environment variables:
- DATABASE_URL: Invalid url
- AUTH_SECRET: AUTH_SECRET is required in production

```

A junior engineer can read that and fix it without grep-splunking.

12. Tests — three levels, three tools

Level	Tool	What we test	Speed
Unit	Vitest	Pure functions: maths ops, Zod schemas, helpers	< 5 seconds
Smoke	Vitest	Same, plus the comment Markdown-to-HTML pipeline	< 5 seconds
Browser	Playwright	Real Chromium driving real pages against <code>pnpm dev</code>	~50 seconds
A11y	axe-core	WCAG violations on three key pages	~30 seconds

12.1 Unit tests

Vitest runs `tests/unit/**/*.test.ts`. Each file imports a pure module and asserts on its return value:

```
import { describe, expect, it } from "vitest";
import { renderCommentHtml } from "@lib/comments-render";

describe("renderCommentHtml", () => {
  it("renders Markdown to HTML", () => {
    const html = renderCommentHtml("**bold** and *italic*");
    expect(html).toMatch(/<strong>bold<\strong>/);
    expect(html).toMatch(/<em>italic<\em>/);
  });
});
```

Pure functions are *easy* to test because there's no setup. Isolating "the bit that has logic" from "the bit that talks to the network or disk" is a habit worth practicing — every time you do it, the tested piece becomes both safer and easier to reason about.

We require **100% line coverage on `src/lib/transformer/`** because that's where the maths lives. The threshold is enforced in CI; a PR that drops coverage below 100% fails to merge.

12.2 Browser tests

Playwright spawns a real headless Chromium, hits `pnpm dev`, and drives the UI:

```
test("signed-in user can post a comment and progress is tracked", async ({ page }) => {
  await signInAs(page, "carol");
  await page.goto(`/learn/01-overview`);

  const body = `Hello from Phase 8 - ${Date.now()}`;
  await page.getByLabel(/comment body/i).fill(body);
  await page.getByRole("button", { name: /^Post$/ }).click();

  await expect(page.getByText(body)).toBeVisible();
});
```

`signInAs(page, "carol")` uses our **E2E credentials provider** — a test-only Auth.js provider that lets us sign in with a fake username, no real GitHub round-trip. It's gated by `E2E_TEST_AUTH=true`, and the production deploy never sets that, so the credentials provider can't be used to bypass GitHub OAuth on the live site.

12.3 Accessibility tests

`tests/e2e/a11y.spec.ts` uses `@axe-core/playwright` to scan our key pages for WCAG violations:

```
const results = await new AxeBuilder({ page })
  .withTags(["wcag2a", "wcag2aa", "wcag21aa"])
  .analyze();

const blocking = results.violations.filter(v =>
  ["serious", "critical"].includes(v.impact ?? ""),
);
expect(blocking).toEqual([]);
```

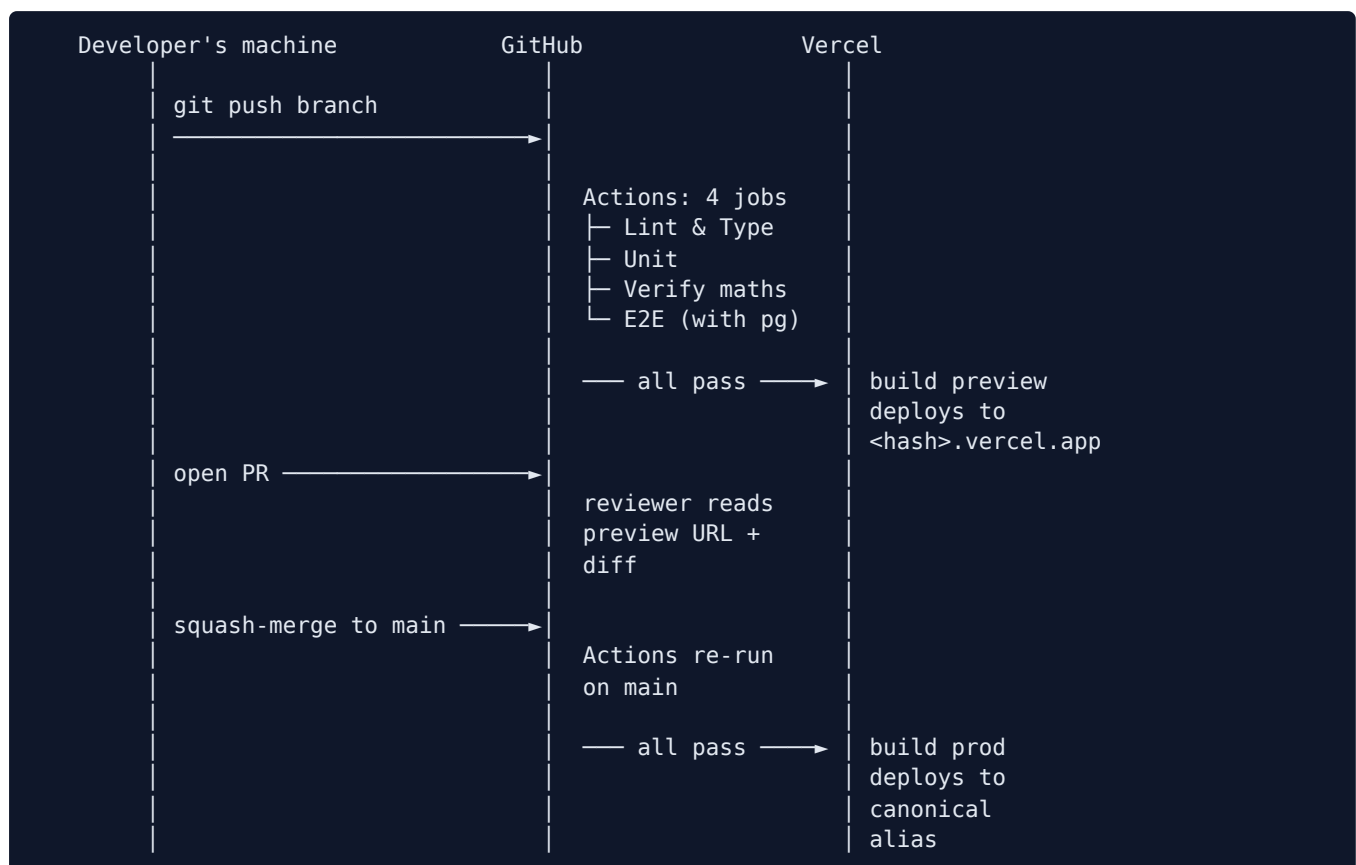
Common things axe catches: missing `alt` on images, low colour contrast, missing form labels, ARIA misuse. Junior engineers often find this is where their UI work is "discovered" to have problems — letting a tool catch them before review is humane.

12.4 Verifying the maths against PyTorch

`scripts/verify-maths.ts` is the fourth and most domain-specific check. The TypeScript transformer ops have to agree numerically with a reference PyTorch implementation. We have the PyTorch reference run once, dump fixtures to JSON, commit them. CI runs the TS ops on the same inputs and asserts every output is within `1e-5` of PyTorch.

This is a worked example of **golden-file testing**: when the answer isn't in your head, capture a known-good output once, re-run against it forever. If the test breaks, you either accept the new output (commit the new fixture) or fix the regression.

13. CI/CD — what happens when you push a branch



`.github/workflows/ci.yml` runs four jobs in parallel on every PR and on `main`:

1. **Lint & Typecheck:** `pnpm lint`, `pnpm typecheck`, `pnpm format:check`.

2. **Unit Tests:** `pnpm test`.
3. **Verify maths against PyTorch fixtures:** `pnpm verify:maths`.
4. **E2E Tests:** spins up a service-container Postgres, runs Playwright.

Each job uses GitHub Actions' `actions/setup-node` and `pnpm/action-setup` to set up the environment, then runs the script. Cache-keyed off the lockfile, so installs are fast.

The E2E job is the most interesting — it has a Postgres sidecar:

```
services:
  postgres:
    image: postgres:16
    env:
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: transformer_explainer_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5
```

GitHub spins up a Postgres container alongside our test job. We point `DATABASE_URL` at `localhost:5432`, run `pnpm db:push` to create the schema, run `pnpm db:seed` to populate it, then run Playwright. Total runtime ~3 minutes.

After CI passes, we merge to `main`. **Vercel watches the `main` branch** and triggers a production build automatically (when the GitHub integration is connected). Within ~90 seconds the new code is live at `transformer-explainer-three.vercel.app`.

Preview deployments are also triggered for every PR — each gets its own URL like `transformer-explainer-
<random>.vercel.app`. You can poke at your branch's behaviour without merging.

This continuous-delivery loop — push code → CI runs → merge → live in under five minutes — is the modern web's superpower. It only works if your CI is *trustworthy*: both fast enough that you don't ignore it, and strict enough that "CI is green" actually means "this is safe to ship".

14. Deploy specifics — what Vercel does with our code

Knowing your hosting provider's mental model is part of being production-effective. Vercel's looks like this:

14.1 The build

`vercel deploy` packages the working directory and ships it to Vercel's build infrastructure. Vercel detects the framework (Next.js), runs `pnpm install` and `pnpm build`, and produces the **build output**:

- A folder of static files (`/_next/static/*` — JS bundles, CSS, fonts). These are served straight from the global CDN.
- One serverless function per dynamic route. `app/learn/[slug]/page.tsx` becomes `/learn/[slug]` function. `app/api/events/route.ts` becomes `/api/events`.
- A routing manifest mapping URL patterns to functions or static files.

14.2 Two file types, two delivery paths

File type	Path	Delivery
Static asset	<code>/_next/static/</code>	Edge CDN, cached forever
Static page	<code>/some/page</code>	Edge CDN, may revalidate
Dynamic page	<code>/learn/[slug]</code>	Serverless function on demand
Route handler	<code>/api/...</code>	Serverless function on demand

Our pages use `dynamic = "force-dynamic"` so they're always run as functions (we need fresh session lookups, fresh progress data). If a page were truly static (a marketing page) we could let Vercel serve it from the CDN — much faster.

14.3 The cold-start tax

The first request after a deploy or after long idleness pays a cold start: Vercel has to load our code into a fresh Node process. We measured ~200-500 ms in this project. Mitigations:

- **Keep the bundle small.** Big function bundles cold-start slowly. Don't import a giant library to use one function.
- **Don't do work at module load.** Lazy initialisation, like our DOMPurify trick.
- **Warm with traffic.** The serverless function stays warm for a few minutes after a request.

14.4 Logs

Vercel collects every `console.log`, `console.error`, and HTTP request into runtime logs. The `vercel logs <deployment>` CLI tails them. The MCP server's `get_runtime_logs` tool surfaces them with filtering. You will read these constantly when chasing production bugs — get comfortable with them.

In our codebase, the `runOrFallback` helper deliberately rate-limits log output (one per minute per call site) so a sustained outage doesn't flood Vercel's log retention.

14.5 Environment variables

Vercel stores env vars per environment (production / preview / development). Plain CLI:

```
vercel env add DATABASE_URL production    # prompts for value
vercel env ls production                  # list them
```

In a real team you'd integrate with a secret manager (Doppler, AWS Secrets Manager, Vault). For a portfolio site the dashboard / CLI is plenty.

15. Practical lessons — the bugs we hit and what they teach

This section is candid: every bug in the **deploy** phase is one we actually hit, in order.

15.1 DATABASE_URL is required for drizzle-kit

`drizzle-kit` doesn't auto-load `.env.local`. Only Next.js does. So `pnpm db:push` and `pnpm db:seed` errored with no obvious cause when run from a fresh clone.

Fix: load `.env.local` explicitly at the top of `drizzle.config.ts` and `scripts/seed-db.ts` via the `dotenv` package.

Lesson: every script you ship that reads env should explicitly declare what it needs and where it gets it from. Don't rely on the ambient environment of one tool to contaminate another.

15.2 Dev console spammed with `MissingSecret` and 500 errors on a fresh clone

A user following our quickstart got a flood of Auth.js errors and 500s on `/api/events` because the placeholder env values pointed at a non-existent host (`host.neon.tech`).

Fix: introduce `runOrFallback` and wrap every DB call site that's on the *page-paint* path. The result: a fresh clone with placeholder env renders `/learn/*` and `/playground` cleanly; only the auth/DB features no-op silently.

Lesson: degrade gracefully on the read path. Don't leak infrastructure failures to end users when a sensible default exists.

15.3 Vercel's security scanner blocking the deploy

First Vercel deploy failed with: *Vulnerable version of next-mdx-remote detected (5.0.0)*. Vercel runs `npm audit` - equivalent checks before deploy and refuses known-CVE packages.

Fix: `pnpm add next-mdx-remote@latest` (6.0.0 was API-compatible for our usage), local build/test, redeploy.

Lesson: the platform you deploy to may impose its own security gates. That's almost always good; just don't be surprised by them.

15.4 `OAuthCallbackError` on first GitHub sign-in

Postgres rejected the user insert because `mapGitHubProfile` returned `id: "<numeric github id>"` and the column was `uuid`.

Fix: drop `id` from the mapper, let `defaultRandom()` generate a real UUID.

Lesson: TypeScript types are not database constraints. The compiler believed `id: string` was fine; Postgres disagreed. Always validate end-to-end against your storage.

15.5 `ERR_TOO_MANY_REDIRECTS` on the production sign-in

Auth.js v5: setting `pages.signIn` to `/api/auth/signin` (the default handler path) creates a redirect loop. It's supposed to point at a **custom** UI page if you provide one.

Fix: remove the field.

Lesson: read the framework's docs for the exact field semantics, not just the type signature. v4 and v5 changed this behaviour.

15.6 `OAuthAccountNotLinked` after seeding

`db:seed` had created an admin user row with the right email and a placeholder GitHub id. The first real OAuth sign-in matched on email but couldn't link the GitHub identity, so Auth.js refused.

Fix: `allowDangerousEmailAccountLinking: true` on the GitHub provider. Safe because GitHub verifies emails — a hostile party can't register `your.email@...` on github.com.

Lesson: secure-by-default settings sometimes don't match your specific bootstrap flow. Read what the flag actually checks before disabling it; don't disable security flags casually.

These six bugs all hit only **after deploy**. They're a microcosm of why production logging, blast-radius-aware deployment (preview → production), and fast-iteration toolchains matter so much in real-world backend work.

16. What's missing — things you'd add for real production

This is a portfolio-grade demo, not a real product. Here are the gaps a real production system would close, roughly in order:

1. **Distributed rate limiting**. The token bucket is per-instance; real protection needs a shared store (Redis, Upstash, Edge Config).
2. **Background jobs**. Anything time-consuming should go off the request path. Options: Vercel Cron, Inngest, BullMQ + Redis.
3. **Email**. Auth.js can do passwordless email auth; that needs a transactional email provider (Resend, SendGrid, Postmark).
4. **Observability**. We have request logs and a homemade analytics funnel. A real product wants tracing (OpenTelemetry → Honeycomb, Datadog) and alerts (Sentry, PagerDuty).
5. **Schema migrations** (not `db:push`). Generated, versioned SQL files that you apply deterministically.
6. **Multi-environment Postgres**. Separate dev/staging/prod databases. Neon makes this cheap (branches off main).
7. **Secrets rotation**. AUTH_SECRET, OAuth client secrets, DB passwords should be rotated on a schedule. Plumbing this without downtime takes some care.
8. **Authorisation beyond admin/non-admin**. RBAC, scoped tokens for API access, audit logs of who changed what.
9. **Rate limiting on the *paid* operations** (compute), not just on analytics.
10. **A CDN for the screenshots and PDFs**. Right now they're served by Vercel directly.

If you can articulate the *why* behind each of these, you're past the junior bar.

17. Reading the codebase top-down

If you want to learn from this repo, here's the recommended reading order:

1. **README.md**. Sets the stage.
2. **SPEC.md**. The original design document. Skim §1-3 for goals, §4-7 for the user-visible scope.
3. **src/lib/env.ts**. The boundary between OS env and the app.
4. **src/lib/db/schema.ts**. Read the table definitions and trace the foreign keys.
5. **src/lib/db/client.ts**. The connection pool pattern.
6. **src/lib/auth/{config,helpers,index}.ts**. The Auth.js wiring.
7. **src/lib/transformer/**. One file per maths op. The contract pattern (every op accepts an optional `Trace`) is the heart of the "see real values" design. (This is the *content*; you can skip if you only care about the backend pattern.)
8. **src/app/learn/[slug]/page.tsx**. A representative dynamic page.
9. **src/app/api/sections/[slug]/comments/route.ts**. A representative auth + DB endpoint.
10. **src/app/api/events/route.ts**. The graceful-degradation pattern.
11. **tests/unit/transformer/attention.test.ts**. How to write a purely numerical test.

12. `tests/e2e/comments-progress.spec.ts` . How to write an integration test that sets up state, performs an action, and asserts on observable consequences.
13. `PROGRESS.md` . Phase-by-phase log of what was built and why, with deviations and follow-ups noted at each step. Hard to find in a real codebase; treasure it when you do.
14. `.github/workflows/ci.yml` . The CI definition.

Once you've read those in order, open the production site, sign in, post a comment, save an experiment. Watch the network panel in your browser dev-tools as you do — you'll see every endpoint we just walked through fire in real time. That feedback loop, between code you've read and behaviour you can directly probe, is the fastest way to internalise a system.

Appendix A — Useful commands cheat-sheet

```
# Local development
pnpm dev           # http://localhost:3000
pnpm test          # Vitest unit tests
pnpm test:coverage # ...with thresholds enforced
pnpm test:e2e      # Playwright (boots dev server itself)
pnpm verify:maths  # cross-check TS ops vs. PyTorch fixtures

pnpm lint          # ESLint
pnpm typecheck     # tsc --noEmit
pnpm format:check  # Prettier --check
pnpm format        # Prettier --write

pnpm db:push       # apply schema to the DB in $DATABASE_URL
pnpm db:seed       # idempotent seed
pnpm db:studio     # open Drizzle Studio (a DB GUI)

# Vercel
vercel link        # link this dir to a Vercel project
vercel env ls      # list env vars
vercel env add NAME prod # add an env var (will prompt for value)
vercel deploy --prod # deploy current dir to production
vercel logs <url>  # tail runtime logs

# Git
gh pr create --fill # open a PR with the commit body as description
gh pr checks <pr>   # see CI status
gh pr merge <pr> --squash # squash-merge
```

Appendix B — Glossary

- **App Router**. The newer Next.js routing system (introduced in v13) that uses the `app/` directory and supports Server Components.
- **CSRF**. Cross-Site Request Forgery. An attacker tricks a logged-in user's browser into making an unwanted request to your site. Mitigated by a single-use token paired between cookie and form.
- **Cold start**. The latency cost of spinning up a fresh function instance the first time it's needed.
- **Drizzle adapter**. The piece of Auth.js that knows how to read/write user/account/session rows via Drizzle.
- **Fluid Compute**. Vercel's name for their serverless model where one warm function instance handles many concurrent requests instead of spinning up a new one per request.

- **Hot reload.** The dev server reloads modules as you save files, preserving state where possible.
- **Hydrate.** The process of attaching React event handlers to server-rendered HTML on the client.
- **JSDOM.** A browser-DOM emulator that runs in Node.js. Used by `isomorphic-dompurify`.
- **JWT.** JSON Web Token. A signed, base64-encoded JSON object; self-contained but non-revocable.
- **MDX.** Markdown with inline JSX. Lets us mix prose and components.
- **OAuth callback.** The URL on your site that the OAuth provider redirects the user back to with an authorisation code.
- **Postgres.** The open-source relational database, version 16-17 [here](#).
- **Server Component.** A React component that runs only on the server and never ships JS to the browser.
- **Token bucket.** A rate-limiting algorithm where each call costs a token and tokens refill over time.
- **UUID.** Universally Unique Identifier. A 128-bit value, conventionally written as `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`. Effectively collision-free for any practical scale.
- **Zod.** A runtime schema validator with TypeScript inference.

End of report.